

C++



```
#include <iostream>
#include <vector>
#include <string>
#include <queue>
#include <unordered_map>

std::vector<std::string> taskScheduler(std::vector<std::string>& tasks, int n) {
    std::unordered_map<std::string, int> counts;
    for (auto& t : tasks) counts[t]++;
    std::priority_queue<std::pair<int, std::string>> maxHeap;
    for (auto& it : counts) maxHeap.push({it.second, it.first});
    std::queue<std::pair<int, std::pair<std::string, int>>> q;
    std::vector<std::string> res;
    int time = 0;
    while (!maxHeap.empty() || !q.empty()) {
        time++;
        if (!maxHeap.empty()) {
            auto top = maxHeap.top(); maxHeap.pop();
            res.push_back(top.second);
            if (top.first - 1 > 0) q.push({top.first - 1, {top.second, time + n}});
        } else res.push_back("NOTHING");
        if (!q.empty() && q.front().second.second == time) {
            auto f = q.front(); q.pop();
            maxHeap.push({f.first, f.second.first});
        }
    }
    return res;
}
```

Python

```
def solve(n):
    if n == 0: return ""
    # Try replacing only the last digit
    for i in range(1, 10):
        s = (n - 1) + (i**2)
        if int(s**0.5)**2 == s:
            return '1' * (n - 1) + str(i)
    # Try replacing the last two digits
    for i in range(1, 10):
        for j in range(1, 10):
            s = (n - 2) + (i**2) + (j**2)
            if int(s**0.5)**2 == s:
                return '1' * (n - 2) + str(i) + str(j)
```

Java :

```
public class Solution {
    public static String solve(int n) {
        for (int i = 1; i <= 9; i++) {
            long s = (n - 1) + (i * i);
            if (Math.pow((int) Math.sqrt(s), 2) == s)
                return "1".repeat(n - 1) + i;
        }
        for (int i = 1; i <= 9; i++) {
            for (int j = 1; j <= 9; j++) {
                long s = (n - 2) + (i * i) + (j * j);
                if (Math.pow((int) Math.sqrt(s), 2) == s)
                    return "1".repeat(n - 2) + i + "" + j;
            }
        }
        return "";
    }
}
```

C++ :

```
#include <iostream>
#include <string>
#include <cmath>

std::string solve(int n) {
    for (int i = 1; i <= 9; i++) {
        long long s = (n - 1) + (i * i);
        long long root = sqrt(s);
        if (root * root == s) return std::string(n - 1,
'1') + std::to_string(i);
    }
    for (int i = 1; i <= 9; i++) {
        for (int j = 1; j <= 9; j++) {
            long long s = (n - 2) + (i * i) + (j * j);
            long long root = sqrt(s);
            if (root * root == s) return std::string(n -
2, '1') + std::to_string(i) + std::to_string(j);
        }
    }
    return "";
}
```

C++

```
#include <iostream>
#include <string>

std::string customCaesarCipher(std::string text, int key) {
    if (key < 0) return "INVALID INPUT";

    for (int i = 0; i < text.length(); i++) {
        if (text[i] >= 'a' && text[i] <= 'z') {
            text[i] = (text[i] - 'a' + key) % 26 + 'a';
        }
        else if (text[i] >= 'A' && text[i] <= 'Z') {
            text[i] = (text[i] - 'A' + key) % 26 + 'A';
        }
        else if (text[i] >= '0' && text[i] <= '9') {
            text[i] = (text[i] - '0' + key) % 10 + '0';
        }
    }
    return text;
}
```

Python

```
def custom_caesar_cipher(text, key):
    if key < 0:
        return "INVALID INPUT"

    result = []
    for char in text:
        if 'a' <= char <= 'z':
            # Shift lowercase: (current - 'a' + key) % 26 + 'a'
            result.append(chr((ord(char) - ord('a') + key) % 26 + ord('a')))
        elif 'A' <= char <= 'Z':
            # Shift uppercase: (current - 'A' + key) % 26 + 'A'
            result.append(chr((ord(char) - ord('A') + key) % 26 + ord('A')))
        elif '0' <= char <= '9':
            # Shift digits: (current - '0' + key) % 10 + '0'
            result.append(chr((ord(char) - ord('0') + key) % 10 + ord('0')))
        else:
            # Leave symbols/spaces unchanged
            result.append(char)

    return "".join(result)
```

Q. Implement a custom Caesar Cipher that shifts both alphabetic characters (A-Z, a-z) and numeric digits (0-9) by a given key. Alphabets wrap around within 26 characters, and digits wrap around within 10. Symbols like "-" remain unchanged.

Example 1:

- Input: Text: "All the best", Key: 1
- Output: "Bmm uif gftu"

Example 2:

- Input: Text: "5-abc", Key: 2
- Output: "7-cde"

Example 3:

- Input: Text: "Hello", Key: -1
- Output: "INVALID INPUT"

Logic :

- Validation: If the key is less than 0, return "INVALID INPUT".
- Alphabets: For each letter, determine its base ('a' or 'A'). Shift it using $(\text{char} - \text{base} + \text{key}) \% 26 + \text{base}$.
- Digits: For each digit, shift it using $(\text{char} - '0' + \text{key}) \% 10 + '0'$.
- Constants: Non-alphanumeric characters are appended to the result without modification.



Q. Search for a specific substring within a main string. If the substring exists, return the index of its first occurrence; otherwise, return -1.

Example 1:

- Input: String: "takeuforward", Substring: "forward"
- Output: 5
- Explanation: The substring "forward" starts at index 5 of "takeuforward".

Example 2:

- Input: String: "hello", Substring: "world"
- Output: -1

Example 3:

- Input: String: "programming", Substring: "gram"
- Output: 3

Logic :

- Linear Search: Iterate through the main string from index 0 to N-M (where N is main string length and M is substring length).
- Match: At each index, check if the next M characters match the substring.
- Optimization: Use built-in library functions like `find()` or `indexOf()` which are highly optimized for searching.



Q. Given a string, convert all its uppercase characters to lowercase and all its lowercase characters to uppercase.

Example 1:

- Input: "PlaCito"
- Output: "pLAcITO"

Example 2:

- Input: "JAVA is FUN"
- Output: "java IS fun"

Example 3:

- Input: "123 abc"
- Output: "123 ABC"

Logic :

- Iterate: Traverse through the string character by character.
- Check Case: For each character, check if it is uppercase or lowercase.
- Swap: If it's uppercase, convert to lowercase using `tolower()`. If it's lowercase, convert to uppercase using `toupper()`.
- Non-alphabetic: Ignore numbers or symbols and keep them as they are.



Java

```
public String highestRepeated(String s) {  
    String[] words = s.split(" ");  
    String res = "";  
    int maxFreq = 0;  
    for (String w : words) {  
        int[] freq = new int[256];  
        int localMax = 0;  
        for (char c : w.toCharArray()) {  
            freq[c]++;  
            localMax = Math.max(localMax, freq[c]);  
        }  
        if (localMax > maxFreq && localMax > 1) {  
            maxFreq = localMax;  
            res = w;  
        }  
    }  
    return res;  
}
```

Q. Given a string, find the word that contains the highest frequency of any single character. If no word has repeated letters, return the first word or a specific indicator.

Example 1:

- Input: "abcdef gghhkkk apple"
- Output: "gghhkkk"
- Explanation: "gghhkkk" has 'k' repeated 3 times, which is the highest in this string.

Example 2:

- Input: "hello world"
- Output: "hello" ('l' is repeated twice)

Example 3:

- Input: "no repeats here"
- Output: "repeats" ('e' is repeated twice)

Logic :

- Word Extraction: Split the string into individual words.
- Character Counting: For each word, use a frequency map (or array) to count how many times each character appears within that specific word.
- Find Max in Word: Determine the highest frequency of any single character in that word.
- Global Max: Keep track of the word that yielded the highest frequency across the entire string.



Q. Given a string, count the number of occurrences of each character present in it.

Example 1:

- Input: "apple"
- Output: a:1, p:2, l:1, e:1

Example 2:

- Input: "aba"
- Output: a:2, b:1

Example 3:

- Input: "zzz"
- Output: z:3

Logic :

- Map: Use a Hash Map (Dictionary) where the key is the character and value is the count.
- Count: Iterate through the string; if a character exists in the map, increment its value; otherwise, add it with a value of 1.



C++

```
int sumDigits(string s) {  
    int sum = 0;  
    for(char c : s) {  
        if(isdigit(c)) sum += (c - '0');  
    }  
    return sum;  
}
```

Python

```
def sum_digits(s):  
    return sum(int(c) for c in s if c.isdigit())
```

Java

```
public int sumDigits(String s) {  
    int sum = 0;  
    for(char c : s.toCharArray()) {  
        if(Character.isDigit(c)) sum += Character.getNumericValue(c);  
    }  
    return sum;  
}
```

C++

```
string removeBrackets(string s) {  
    string res = "";  
    for(char c : s) {  
        if(c != '(' && c != ')') res += c;  
    }  
    return res;  
}
```

Python

```
def remove_brackets(s):  
    return "".join([c for c in s if c not in '()'])
```

Java

```
public String removeBrackets(String s) {  
    return s.replaceAll("[()]", "");  
}
```

Q. Convert a given binary number (base-2) into its equivalent octal (base-8) representation.

Example 1:

- Input: 110011
- Output: 63
- Explanation: Group digits into threes from right to left: (110)(011).
110 = 6 and 011 = 3.

Example 2:

- Input: 1010
- Output: 12

Example 3:

- Input: 11111
- Output: 37

Logic :

- **Decimal First:** Convert the binary number into a decimal number.
- **Octal Conversion:** Repeatedly divide the decimal number by 8.
- **Collect Remainders:** The remainders, read in reverse order, form the octal number.
- **Shortcut:** Alternatively, group binary digits in sets of three from right to left and convert each group directly.



Q. Given coefficients a , b , and c of a quadratic equation $ax^2 + bx + c = 0$, find all possible roots.

Example 1:

- Input: $a=1, b=-5, c=6$
- Output: Roots: 3, 2

Example 2:

- Input: $a=1, b=-2, c=1$
- Output: Roots: 1, 1

Example 3:

- Input: $a=1, b=1, c=1$
- Output: Complex Roots

Logic :

- Discriminant: Calculate $D = b^2 - 4ac$.
- Condition 1: If $D > 0$, roots are real and distinct: $(-b \pm \sqrt{D}) / 2a$.
- Condition 2: If $D = 0$, roots are real and equal: $-b / 2a$.
- Condition 3: If $D < 0$, roots are imaginary.



Java

```
public static long replaceZeros(long n) {  
    if (n == 0) return 1;  
  
    long res = 0;  
    long weight = 1;  
    long temp = n;  
  
    while (temp > 0) {  
        long digit = temp % 10;  
        if (digit == 0) {  
            digit = 1;  
        }  
        res += digit * weight;  
        weight *= 10;  
        temp /= 10;  
    }  
  
    return res;  
}
```

C++

```
bool isStrongNumber(int n) {  
    // Static ensures the array is only created once in memory  
    static const int f[] = {1, 1, 2, 6, 24, 120, 720, 5040, 40320, 362880};  
  
    int temp = n;  
    int totalSum = 0;  
  
    while (temp > 0) {  
        totalSum += f[temp % 10];  
        // Optional optimization: exit early if sum exceeds n  
        if (totalSum > n) return false;  
        temp /= 10;  
    }  
  
    return totalSum == n;  
}
```

Python

```
def is_strong_number(n):  
    # Lookup table for factorials of 0-9  
    f = [1, 1, 2, 6, 24, 120, 720, 5040, 40320, 362880]  
  
    temp = n  
    total_sum = 0  
  
    while temp > 0:  
        total_sum += f[temp % 10]  
        temp //= 10  
  
    return total_sum == n
```

Java

```
public static boolean isStrongNumber(int n) {  
    // Precomputed factorials for 0 through 9  
    final int[] f = {1, 1, 2, 6, 24, 120, 720, 5040, 40320, 362880};  
  
    int temp = n;  
    int totalSum = 0;  
  
    while (temp > 0) {  
        totalSum += f[temp % 10];  
        temp /= 10;  
    }  
  
    return totalSum == n;  
}
```

C++

```
void primeFactors(int n) {  
    while (n % 2 == 0) { cout << 2 << " "; n /= 2; }  
    for (int i = 3; i * i <= n; i += 2) {  
        while (n % i == 0) { cout << i << " "; n /= i; }  
    }  
    if (n > 2) cout << n;  
}
```

Python

```
def prime_factors(n):  
    i = 2  
    factors = []  
    while i * i <= n:  
        while n % i == 0:  
            factors.append(i)  
            n //= i  
        i += 1  
    if n > 1: factors.append(n)  
    return factors
```

Java

```
public static void primeFactors(int n) {  
    while (n % 2 == 0) { System.out.print(2 + " "); n /= 2; }  
    for (int i = 3; i * i <= n; i += 2) {  
        while (n % i == 0) { System.out.print(i + " "); n /= i; }  
    }  
    if (n > 2) System.out.print(n);  
}
```

Q. Given a number N, find and print all its prime factors. Prime factors are the prime numbers that multiply together to give the original number.

Example 1:

- Input: N = 60
- Output: 2, 2, 3, 5

Example 2:

- Input: N = 13
- Output: 13

Example 3:

- Input: N = 90
- Output: 2, 3, 3, 5

Logic :

- **Handle 2:** While N is divisible by 2, print 2 and divide N by 2.
- **Odd Factors:** Loop from $i = 3$ to $\sqrt{N}$ with a step of 2.
- **Divide:** If i divides N, print i and divide N by i repeatedly.
- **Remaining:** If N is still greater than 2, the remaining N is also a prime factor.



Q. A washing machine estimates time based on weight: 0g = 0 mins, 1-2000g = 25 mins, 2001-4000g = 35 mins, 4001-7000g = 45 mins. If weight exceeds 7000, it is "OVERLOADED".

Example 1:

- Input: 2000
- Output: Time Estimated: 25 minutes

Example 2:

- Input: 0
- Output: Time Estimated: 0 minutes

Example 3:

- Input: 8000
- Output: OVERLOADED

Logic :

- Use conditional if-else blocks to categorize the input weight into the specified ranges.
- Validate for negative inputs (INVALID INPUT).



Q. An automobile company manufactures both Two-Wheelers (TW) and Four-Wheelers (FW). Given the total number of vehicles (V) and the total number of wheels (W), find the exact number of Two-Wheelers and Four-Wheelers to manufacture.

Example 1:

- Input: $V = 200$, $W = 540$
- Output: $TW = 130$, $FW = 70$
- Explanation: $130 + 70 = 200$ vehicles. $(70 \times 4) + (130 \times 2) = 540$ wheels.

Example 2:

- Input: $V = 100$, $W = 200$
- Output: $TW = 100$, $FW = 0$

Example 3:

- Input: $V = 5$, $W = 100$
- Output: INVALID INPUT (Wheels cannot exceed $4 \times V$).

Logic :

- Let x be Two-Wheelers and y be Four-Wheelers.
- Equations: $x + y = V$ and $2x + 4y = W$.
- Solving for y : $2(V - y) + 4y = W \implies 2V + 2y = W \implies y = (W - 2V) / 2$.
- Constraints: W must be even, $W \geq 2$, and $W \leq 4V$.



Java

```
public class PalindromeSplitter {
    public static boolean isPalindrome(String s) {
        int left = 0, right = s.length() - 1;
        while (left < right) {
            if (s.charAt(left++) != s.charAt(right--)) return false;
        }
        return true;
    }

    public static void findSplit(String s) {
        int n = s.length();
        for (int i = 1; i <= n - 2; i++) {
            String first = s.substring(0, i);
            if (isPalindrome(first)) {
                for (int j = i + 1; j <= n - 1; j++) {
                    String second = s.substring(i, j);
                    String third = s.substring(j);

                    if (isPalindrome(second) && isPalindrome(third)) {
                        System.out.println(first);
                        System.out.println(second);
                        System.out.println(third);
                        return;
                    }
                }
            }
        }
        System.out.println("Impossible");
    }
}
```

C++



```
#include <iostream>
#include <string>
#include <algorithm>

bool isPalindrome(std::string s) {
    std::string rev = s;
    std::reverse(rev.begin(), rev.end());
    return s == rev;
}

void splitThreePalindromes(std::string s) {
    int n = s.length();

    for (int i = 1; i <= n - 2; i++) {
        std::string first = s.substr(0, i);
        if (isPalindrome(first)) {
            for (int j = 1; i + j <= n - 1; j++) {
                std::string second = s.substr(i, j);
                std::string third = s.substr(i + j);

                if (isPalindrome(second) && isPalindrome(third)) {
                    std::cout << first << "\n" << second << "\n" << third << std::endl;
                    return;
                }
            }
        }
    }

    std::cout << "Impossible" << std::endl;
}
```

C++

```
#include <vector>
#include <unordered_set>
#include <algorithm>

int longestSuccessiveSequence(std::vector<int>& nums) {
    std::unordered_set<int> num_set(nums.begin(), nums.end());
    int max_streak = 0;

    for (int num : num_set) {
        // Check if 'num' is the start of a sequence
        if (num_set.find(num - 1) == num_set.end()) {
            int current_num = num;
            int current_streak = 1;

            while (num_set.find(current_num + 1) != num_set.end()) {
                current_num += 1;
                current_streak += 1;
            }

            max_streak = std::max(max_streak, current_streak);
        }
    }
    return max_streak;
}
```

Python

```
def longest_successive_sequence(nums):
    if not nums:
        return 0

    num_set = set(nums)
    max_streak = 0

    for num in num_set:
        # Only start counting if 'num' is the beginning of a sequence
        if (num - 1) not in num_set:
            current_num = num
            current_streak = 1

            while (current_num + 1) in num_set:
                current_num += 1
                current_streak += 1

            max_streak = max(max_streak, current_streak)

    return max_streak
```

Q. Given an integer array A of size N , return the length of the longest successive sequence. A sequence is successive when adjacent elements have an absolute difference of 1.

Example 1:

- Input: $A = [5, 8, 3, 2, 1, 4]$, $N = 6$
- Output: 5
- Explanation: The sequence is $[1, 2, 3, 4, 5]$.

Example 2:

- Input: $A = [1, 9, 3, 10, 4, 20, 2]$
- Output: 4 ($[1, 2, 3, 4]$)

Example 3:

- Input: $A = [10, 5, 100]$
- Output: 1

Logic :

- Set for Lookup: Store all numbers in a Hash Set for $O(1)$ search.
- Identify Start: Iterate through the array. A number x is the start of a sequence if $x-1$ is not in the set.
- Count: If it's a start, keep checking for $x+1$, $x+2$...and track the length. Return the maximum length found



C++

```
void countVCS(string s) {  
    int v = 0, c = 0, sp = 0;  
    for (char ch : s) {  
        ch = tolower(ch);  
        if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u') v++;  
        else if (ch >= 'a' && ch <= 'z') c++;  
        else if (ch == ' ') sp++;  
    }  
    cout << "Vowels: " << v << " Consonants: " << c << " Spaces: " << sp;  
}
```

Python

```
def count_vcs(s):  
    vowels = "aeiou"  
    v = sum(1 for char in s.lower() if char in vowels)  
    sp = s.count(' ')  
    c = sum(1 for char in s.lower() if char.isalpha() and char not in vowels)  
    return v, c, sp
```

Java

```
public void countVCS(String s) {  
    int v = 0, c = 0, sp = 0;  
    String vowels = "aeiouAEIOU";  
    for (char ch : s.toCharArray()) {  
        if (vowels.indexOf(ch) != -1) v++;  
        else if (Character.isLetter(ch)) c++;  
        else if (ch == ' ') sp++;  
    }  
}
```

Q. Given a string, write a program to count the total number of vowels, consonants, and white spaces present in it.

Example 1:

- Input: "Hello World"
- Output: Vowels: 3, Consonants: 7, Spaces: 1

Example 2:

- Input: "Java 101"
- Output: Vowels: 2, Consonants: 2, Spaces: 1

Example 3:

- Input: "aeiou"
- Output: Vowels: 5, Consonants: 0, Spaces: 0

Logic :

- Iterate: Loop through each character of the string.
- Identify: Convert the character to lowercase. Check if it is a vowel (a, e, i, o, u), a space, or an alphabetic consonant.
- Counter: Maintain three separate integer variables to store the counts.



Q. Given an octal number (base-8), find its decimal (base-10) equivalent.

Example 1:

- Input: 63
- Output: 51
- Explanation: $(6 \times 8^1) + (3 \times 8^0) = 48 + 3 = 51$.

Example 2:

- Input: 100
- Output: 64

Example 3:

- Input: 7
- Output: 7

Logic :

- **Power of 8:** Multiply each octal digit by 8 raised to the power of its position index (starting from 0 on the right).
- **Accumulate:** Sum all these individual products.
- **Process:** Iterate through the number digit by digit from right to left.



C++

```
#include <string>
#include <algorithm>

std::string decimalToOctal(int n) {
    if (n == 0) return "0";

    std::string octal = "";
    while (n > 0) {
        octal += std::to_string(n % 8);
        n /= 8;
    }
    // Reverse the string as remainders are collected in LIFO order
    std::reverse(octal.begin(), octal.end());
    return octal;
}
```

Python

```
def decimal_to_octal(n):
    # Built-in oct() returns a string starting with '0o'
    if n == 0: return "0"
    return oct(n)[2:]
```

Java

```
public static String decimalToOctal(int n) {
    // Integer class provides a high-performance conversion method
    return Integer.toOctalString(n);
}
```

Q. Check if a number is an Abundant Number. A number is abundant if the sum of its proper divisors (excluding the number itself) is greater than the number.

Example 1:

- Input: 12
- Output: Yes
- Explanation: Divisors of 12 are 1, 2, 3, 4, 6. Sum is 16. Since $16 > 12$, it is Abundant.

Example 2:

- Input: 18
- Output: Yes (Sum of divisors = 21)

Example 3:

- Input: 7
- Output: No (Sum of divisors = 1)

Logic :

- Sum Divisors: Iterate from 1 to $\sqrt{N}$ and find all divisors.
- Optimization: For every i that divides N , add both i and N/i to the sum (ensure not to add N itself).



C++

```
bool isHarshad(int n) {  
    if (n <= 0) return false;  
  
    int temp = n;  
    int digitSum = 0;  
  
    while (temp > 0) {  
        digitSum += temp % 10;  
        temp /= 10;  
    }  
  
    return (n % digitSum == 0);  
}
```

Python

```
def is_harshad(n):  
    if n <= 0: return False  
  
    temp = n  
    digit_sum = 0  
  
    while temp > 0:  
        digit_sum += temp % 10  
        temp //= 10  
  
    return n % digit_sum == 0
```

Java

```
public static boolean isHarshad(int n) {  
    if (n <= 0) return false;  
  
    int temp = n;  
    int digitSum = 0;  
  
    while (temp > 0) {  
        digitSum += temp % 10;  
        temp /= 10;  
    }  
  
    return n % digitSum == 0;  
}
```

Q. Determine if a given number is a Harshad Number. A Harshad number is an integer that is divisible by the sum of its digits.

Example 1:

- Input: 18
- Output: Yes
- Explanation: Sum of digits is $1+8=9$. Since 18 is divisible by 9, it is Harshad.

Example 2:

- Input: 15
- Output: No (Sum = 6, 15 is not divisible by 6)

Example 3:

- Input: 21
- Output: Yes

Logic :

- **Sum Digits:** Extract each digit and calculate their total sum.
- **Condition:** Check if the original number modulo the sum equals 0.



C++

```
long long gcd(long long a, long long b) {
    while (b) {
        a %= b;
        swap(a, b);
    }
    return a;
}

long long findLCM(long long a, long long b) {
    if (a == 0 || b == 0) return 0;
    a = abs(a); b = abs(b);
    // Divide before multiplying to prevent overflow
    return (a / gcd(a, b)) * b;
}
```

Python

```
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def find_lcm(a, b):
    if a == 0 or b == 0: return 0
    # Use absolute values to handle negative inputs
    a, b = abs(a), abs(b)
    return (a // gcd(a, b)) * b
```

Java

```
public static long gcd(long a, long b) {
    while (b != 0) {
        long temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

public static long findLCM(long a, long b) {
    if (a == 0 || b == 0) return 0;
    a = Math.abs(a); b = Math.abs(b);
    // Optimized calculation to avoid overflow
    return (a / gcd(a, b)) * b;
}
```

Q. Calculate the Least Common Multiple (LCM) of two integers A and B.

Example 1:

- Input: A = 4, B = 6
- Output: 12

Example 2:

- Input: A = 15, B = 20
- Output: 60

Example 3:

- Input: A = 3, B = 7
- Output: 21

Logic :

- Relation: Use the formula: $LCM(a, b) = (a \times b) / GCD(a, b)$.
- Optimization: To avoid integer overflow, divide one of the numbers by the GCD before multiplying by the other: $(a / gcd) \times b$.



Q. Find the *Greatest Common Divisor (GCD)* of two numbers A and B, which is the largest positive integer that divides both numbers exactly.

Example 1:

- Input: A = 12, B = 18
- Output: 6

Example 2:

- Input: A = 7, B = 5
- Output: 1

Example 3:

- Input: A = 20, B = 100
- Output: 20

Logic :

- **Euclidean Algorithm:** Use the property $\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$.
- **Base Case:** When b becomes 0, a is the GCD.
- **Efficiency:** This is the most optimized method with $O(\log(\min(A, B)))$ time complexity.



Q. Given a string s , return true if it is possible to split the string s into three non-empty palindromic substrings. Otherwise, return false.

Example 1:

- Input: $s = \text{"abcbdd"}$
- Output: true
- Explanation: "a", "bcb", "dd" are all palindromes.

Example 2:

- Input: $s = \text{"bcbddxy"}$
- Output: false

Logic :

- **Precompute Palindromes:** Create a 2D DP table $\text{isPal}[i][j]$ that stores whether the substring $s[i\dots j]$ is a palindrome.
- **Double Partition:** Use two nested loops to try all possible split points i and j where $0 < i < j < n-1$.
- **Check Conditions:** For each pair (i, j) , check if $s[0\dots i-1]$, $s[i\dots j]$, and $s[j+1\dots n-1]$ are all palindromes using the precomputed table.
- **Early Exit:** Return true as soon as a valid triplet is found.



Java :

```
public List<Integer> spiralOrder(int[][] matrix) {
    List<Integer> res = new ArrayList<>();
    int top = 0, bottom = matrix.length - 1, left = 0, right =
matrix[0].length - 1;
    while (top <= bottom && left <= right) {
        for (int i = left; i <= right; i++) res.add(matrix[top][i]);
        top++;
        for (int i = top; i <= bottom; i++) res.add(matrix[i][right]);
        right--;
        if (top <= bottom) for (int i = right; i >= left; i--)
res.add(matrix[bottom][i]);
        bottom--;
        if (left <= right) for (int i = bottom; i >= top; i--)
res.add(matrix[i][left]);
        left++;
    }
    return res;
}
```

Q. Given an $m \times n$ matrix, return all elements of the matrix in spiral order.

Example 1:

- Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]
- Output: [1,2,3,6,9,8,7,4,5]

Example 2:

- Input: matrix = [[1,2,3,4],[5,6,7,8],[9,10,11,12]]
- Output: [1,2,3,4,8,12,11,10,9,5,6,7]

Example 3:

- Input: matrix = [[1]]
- Output: [1]

Logic :

- **Boundaries:** Define top, bottom, left, and right boundaries of the matrix.
- **Traverse Right:** Move from left to right along the top row, then increment top.
- **Traverse Down:** Move from top to bottom along the right column, then decrement right.
- **Traverse Left:** Move from right to left along the bottom row, then decrement bottom.
- **Traverse Up:** Move from bottom to top along the left column, then increment left.
- **Loop:** Continue until the boundaries cross each other.



Q. Given an $m \times n$ integer matrix, if an element is 0, set its entire row and column to 0's. Do it in-place.

Example 1:

- Input: matrix = `[[1,1,1],[1,0,1],[1,1,1]]`
- Output: `[[1,0,1],[0,0,0],[1,0,1]]`

Example 2:

- Input: matrix = `[[0,1,2,0],[3,4,5,2],[1,3,1,5]]`
- Output: `[[0,0,0,0],[0,4,5,0],[0,3,1,0]]`

Example 3:

- Input: matrix = `[[1,0]]`
- Output: `[[0,0]]`

Logic :

- **Identify Zeros:** Use the first row and first column of the matrix itself as markers to track which rows/cols should be zeroed.
- **First Pass:** Iterate through the matrix. If `matrix[i][j] == 0`, set `matrix[i][0] = 0` and `matrix[0][j] = 0`.
- **Second Pass:** Iterate from (1,1) to (m,n). If the marker in its first row or column is 0, set the cell to 0.
- **Special Handling:** Use two separate boolean variables to check if the very first row or first column originally contained a zero, and zero them out at the end if necessary.



C++

```
ListNode* reverseList(ListNode* head) {  
    ListNode *prev = NULL, *curr = head;  
    while (curr) {  
        ListNode* next = curr->next;  
        curr->next = prev;  
        prev = curr;  
        curr = next;  
    }  
    return prev;  
}
```

Python

```
def reverseList(head):  
    prev, curr = None, head  
    while curr:  
        nxt = curr.next  
        curr.next = prev  
        prev = curr  
        curr = nxt  
    return prev
```

Java

```
public ListNode reverseList(ListNode head) {  
    ListNode prev = null, curr = head;  
    while (curr != null) {  
        ListNode next = curr.next;  
        curr.next = prev;  
        prev = curr;  
        curr = next;  
    }  
    return prev;  
}
```

Q. Given the head of a singly linked list, reverse the list and return its new head.

Example 1:

- Input: head = [1,2,3,4,5]
- Output: [5,4,3,2,1]

Example 2:

- Input: head = [1,2]
- Output: [2,1]

Example 3:

- Input: head = []
- Output: []

Logic :

- **Initialize Pointers:** Create three pointers: prev (NULL), curr (head), and next (NULL).
- **Iterate:** Loop through the list until curr is NULL.
- **Save Next:** Store the next node: next = curr->next.
- **Reverse Link:** Flip the current node's pointer to face backward: curr->next = prev.
- **Move Forward:** Shift prev to curr and curr to next.
- **Final Step:** Return prev as the new head of the reversed list.



Q. Given an non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining.

Example 1:

- Input: height = [0,1,0,2,1,0,1,3,2,1,2,1]
- Output: 6

Example 2:

- Input: height = [4,2,0,3,2,5]
- Output: 9

Example 3:

- Input: height = [2,0,2]
- Output: 2

Logic :

- Two Pointers: Use left at the start and right at the end.
- Max Heights: Keep track of leftMax and rightMax.
- Iterate: While left < right:
 - If height[left] < height[right]:
 - If height[left] >= leftMax, update leftMax.
 - Else, add leftMax - height[left] to total.
 - Move left forward.
 - Else:
 - If height[right] >= rightMax, update rightMax.
 - Else, add rightMax - height[right] to total.
 - Move right backward.



C++

```
vector<vector<int>> generate(int numRows) {  
    vector<vector<int>> res;  
    for (int i = 0; i < numRows; i++) {  
        vector<int> row(i + 1, 1);  
        for (int j = 1; j < i; j++)  
            row[j] = res[i - 1][j - 1] + res[i - 1][j];  
        res.push_back(row);  
    }  
    return res;  
}
```

Python

```
def generate(numRows):  
    res = [[1]]  
    for i in range(1, numRows):  
        row = [1] * (i + 1)  
        for j in range(1, i):  
            row[j] = res[i-1][j-1] + res[i-1][j]  
        res.append(row)  
    return res
```

Java

```
public List<List<Integer>> generate(int numRows) {  
    List<List<Integer>> res = new ArrayList<>();  
    for (int i = 0; i < numRows; i++) {  
        List<Integer> row = new ArrayList<>();  
        for (int j = 0; j <= i; j++) {  
            if (j == 0 || j == i) row.add(1);  
            else row.add(res.get(i-1).get(j-1) + res.get(i-1).get(j));  
        }  
        res.add(row);  
    }  
    return res;  
}
```

Q. Given an integer numRows, return the first numRows of Pascal's triangle.

Example 1:

- Input: numRows = 5
- Output: `[[1],[1,1],[1,2,1],[1,3,3,1],[1,4,6,4,1]]`

Example 2:

- Input: numRows = 1
- Output: `[[1]]`

Example 3:

- Input: numRows = 3
- Output: `[[1],[1,1],[1,2,1]]`

Logic :

- Start: The first row is always [1].
- Loop: For each subsequent row, the first and last elements are always 1.
- Middle Elements: Each middle element at index j is the sum of elements at $j-1$ and j from the previous row.
- Store: Append each completed row to the triangle list.



C++

```
void nextPermutation(vector<int>& nums) {  
    next_permutation(nums.begin(), nums.end());  
}
```

Python

```
def nextPermutation(nums):  
    i = len(nums) - 2  
    while i >= 0 and nums[i] >= nums[i+1]: i -= 1  
    if i >= 0:  
        j = len(nums) - 1  
        while nums[j] <= nums[i]: j -= 1  
        nums[i], nums[j] = nums[j], nums[i]  
    nums[i+1:] = reversed(nums[i+1:])
```

Java

```
public void nextPermutation(int[] nums) {  
    int i = nums.length - 2;  
    while (i >= 0 && nums[i] >= nums[i + 1]) i--;  
    if (i >= 0) {  
        int j = nums.length - 1;  
        while (nums[j] <= nums[i]) j--;  
        swap(nums, i, j);  
    }  
    reverse(nums, i + 1);  
}
```

Q. Implement next permutation, which rearranges numbers into the lexicographically next greater permutation of numbers. If such an arrangement is not possible, rearrange it as the lowest possible order.

Example 1:

- Input: `nums = [1,2,3]`
- Output: `[1,3,2]`

Example 2:

- Input: `nums = [3,2,1]`
- Output: `[1,2,3]`

Example 3:

- Input: `nums = [1,1,5]`
- Output: `[1,5,1]`

Logic :

- **Find Pivot:** From right to left, find the first index i such that $\text{nums}[i] < \text{nums}[i+1]$.
- **Find Successor:** If pivot exists, find the first element from the right that is greater than $\text{nums}[i]$. Let this be index j .
- **Swap:** Swap $\text{nums}[i]$ and $\text{nums}[j]$.
- **Reverse:** Reverse the elements to the right of index i to get the smallest possible sequence for that part.



SQL:

```
SELECT      s.student_id,      s.student_name,      sub.subject_name,
COUNT(e.subject_name) AS attended_exams
FROM Students s
CROSS JOIN Subjects sub
LEFT JOIN Examinations e ON s.student_id =
e.student_id AND sub.subject_name = e.subject_name
GROUP BY
s.student_id,      sub.subject_name
ORDER      BY      s.student_id,
sub.subject_name;
```

Q. Given two arrays, check if the first array is a subset of the second array.

Example 1:

- Input: arr1 = [1, 3, 4], arr2 = [1, 2, 3, 4, 5]
- Output: True

Example 2:

- Input: arr1 = [6, 7], arr2 = [1, 2, 3, 4]
- Output: False

Example 3:

- Input: arr1 = [1], arr2 = [1]
- Output: True

Logic :

- **Store Master:** Store all elements of the larger array (arr2) in a Hash Set.
- **Iterate Subset:** Loop through each element of the potential subset (arr1).
- **Check Set:** If any element of arr1 is not present in the set of arr2, it is not a subset.
- **Return:** If all elements are found, return True.



C++

```
int findEquilibrium(vector<int>& nums) {  
    int total = 0, left = 0;  
    for (int x : nums) total += x;  
    for (int i = 0; i < nums.size(); i++) {  
        if (left == (total - left - nums[i])) return i;  
        left += nums[i];  
    }  
    return -1;  
}
```

Python

```
def findEquilibrium(nums):  
    total, left = sum(nums), 0  
    for i, x in enumerate(nums):  
        if left == (total - left - x): return i  
        left += x  
    return -1
```

Java

```
public int findEquilibrium(int[] nums) {  
    int total = 0, left = 0;  
    for (int x : nums) total += x;  
    for (int i = 0; i < nums.length; i++) {  
        if (left == (total - left - nums[i])) return i;  
        left += nums[i];  
    }  
    return -1;  
}
```

Q. Find the index such that the sum of elements at lower indices is equal to the sum of elements at higher indices.

Example 1:

- Input: `nums = [1, 7, 3, 6, 5, 6]`
- Output: 3
- Explanation: At index 3 (value 6), Left sum: $1+7+3 = 11$, Right sum: $5+6 = 11$.

Example 2:

- Input: `nums = [1, 2, 3]`
- Output: -1

Example 3:

- Input: `nums = [2, 1, -1]`
- Output: 0

Logic :

- **Total Sum:** Calculate the total sum of the entire array.
- **Iterate:** Traverse the array while maintaining a `left_sum` (starting at 0).
- **Calculate Right Sum:** For any index `i`, the `right_sum = total_sum - left_sum - nums[i]`.
- **Check:** If `left_sum == right_sum`, return index `i`.
- **Update:** Add current element to `left_sum` and move to next.
- **Failure:** If no such index exists, return -1.



C++

```
void rotateRight(vector<int>& nums, int k) {  
    int n = nums.size();  
    k %= n;  
    reverse(nums.begin(), nums.end());  
    reverse(nums.begin(), nums.begin() + k);  
    reverse(nums.begin() + k, nums.end());  
}
```

Python

```
def rotateRight(nums, k):  
    n = len(nums)  
    k %= n  
    nums[:] = nums[n-k:] + nums[:n-k]
```

Java

```
public void rotateRight(int[] nums, int k) {  
    int n = nums.length;  
    k %= n;  
    reverse(nums, 0, n - 1);  
    reverse(nums, 0, k - 1);  
    reverse(nums, k, n - 1);  
}
```

Q. Rotate an array by k elements to the left and to the right.

Example 1 (Right):

- Input: nums = [1, 2, 3, 4, 5], k = 2
- Output: [4, 5, 1, 2, 3]

Example 2 (Left):

- Input: nums = [1, 2, 3, 4, 5], k = 2
- Output: [3, 4, 5, 1, 2]

Example 3:

- Input: nums = [10, 20], k = 1
- Output (Right): [20, 10]

Logic :

- Calculate: For any rotation, find the actual steps needed using $k \% n$.
- Reverse Method (Right):
 - Reverse the entire array.
 - Reverse the first k elements.
 - Reverse the remaining n-k elements.
- Reverse Method (Left):
 - Reverse the first k elements.
 - Reverse the remaining n-k elements.
 - Reverse the entire array.



C++

```
vector<int> frequencySort(vector<int>& nums) {  
    unordered_map<int, int> freq;  
    for (int x : nums) freq[x]++;  
    sort(nums.begin(), nums.end(), [&](int a, int b) {  
        return freq[a] != freq[b] ? freq[a] > freq[b] : a < b;  
    });  
    return nums;  
}
```

Python

```
def frequencySort(nums):  
    from collections import Counter  
    freq = Counter(nums)  
    return sorted(nums, key=lambda x: (-freq[x], x))
```

Java

```
public int[] frequencySort(int[] nums) {  
    Map<Integer, Integer> freq = new HashMap<>();  
    for (int x : nums) freq.put(x, freq.getOrDefault(x, 0) + 1);  
    return Arrays.stream(nums).boxed()  
        .sorted((a, b) -> freq.get(a) != freq.get(b) ? freq.get(b) - freq.get(a) : 0)  
        .mapToInt(i -> i).toArray();  
}
```

Q. Given an array of pairs, find all symmetric pairs. A pair (a, b) is symmetric to (c, d) if $a = d$ and $b = c$.

Example 1:

- Input: pairs = [[1, 2], [3, 4], [2, 1], [4, 3]]
- Output: [[1, 2], [3, 4]]
- Explanation: (1, 2) and (2, 1) are symmetric; (3, 4) and (4, 3) are symmetric.

Example 2:

- Input: pairs = [[1, 5], [2, 3], [5, 1]]
- Output: [[1, 5]]

Example 3:

- Input: pairs = [[1, 2], [2, 3]]
- Output: []

Logic :

- **Initialize Map:** Use a hash map to store first as key and second as value.
- **Iterate:** Loop through each pair (first, second).
- **Check Mirror:** Look for the reverse pair (second, first) in the map.
- **Store Pair:** If found, it's a symmetric pair; add it to the result.
- **Update Map:** If not found, add the current pair (first, second) to the map for future matching.



C++

```
vector<int> findNonRepeating(vector<int>& nums) {  
    unordered_map<int, int> freq;  
    vector<int> res;  
    for (int x : nums) freq[x]++;  
    for (int x : nums) {  
        if (freq[x] == 1) res.push_back(x);  
    }  
    return res;  
}
```

Python

```
def findNonRepeating(nums):  
    freq = {}  
    for x in nums:  
        freq[x] = freq.get(x, 0) + 1  
    return [x for x in nums if freq[x] == 1]
```

Java

```
public List<Integer> findNonRepeating(int[] nums) {  
    Map<Integer, Integer> map = new HashMap<>();  
    for (int x : nums) map.put(x, map.getOrDefault(x, 0) + 1);  
    List<Integer> result = new ArrayList<>();  
    for (int x : nums) {  
        if (map.get(x) == 1) result.add(x);  
    }  
    return result;  
}
```

Q. Remove all duplicate elements from an array that is not sorted.

Example 1:

- Input: nums = [4, 3, 9, 2, 4, 1, 10, 89, 34, 3, 2]
- Output: [4, 3, 9, 2, 1, 10, 89, 34]
- Explanation: All subsequent occurrences of 4, 3, and 2 are removed.

Example 2:

- Input: nums = [1, 2, 1, 2]
- Output: [1, 2]

Example 3:

- Input: nums = [5, 4, 3]
- Output: [5, 4, 3]

Logic :

- **Initialize a Set:** Create an empty Hash Set to keep track of seen elements.
- **Create Result:** Create an empty list/array to store unique numbers.
- **Iterate:** Loop through each element in the input array.
- **Check Set:** If the element is NOT in the set, it is unique.
- **Update:** Add it to the set and also append it to the result list.
- **Repeat:** Continue until all elements are checked.



C++

```
double findMedian(vector<int>& nums) {  
    sort(nums.begin(), nums.end());  
    int n = nums.size();  
    if (n % 2 != 0) return (double)nums[n/2];  
    return (double)(nums[(n-1)/2] + nums[n/2]) / 2.0;  
}
```

Python

```
def findMedian(nums):  
    nums.sort()  
    n = len(nums)  
    if n % 2 != 0:  
        return float(nums[n // 2])  
    return (nums[(n - 1) // 2] + nums[n // 2]) / 2.0
```

Java

```
public double findMedian(int[] nums) {  
    Arrays.sort(nums);  
    int n = nums.length;  
    if (n % 2 != 0) return (double)nums[n/2];  
    return (double)(nums[(n-1)/2] + nums[n/2]) / 2.0;  
}
```

Q. Given an array, find the median value. The median is the middle value in a sorted list of numbers.

Example 1:

- Input: nums = [1, 3, 4, 2, 6, 5, 8]
- Output: 4
- Explanation: Sorted array is [1, 2, 3, 4, 5, 6, 8]. The middle element is 4.

Example 2:

- Input: nums = [2, 4, 1, 3]
- Output: 2.5
- Explanation: Sorted: [1, 2, 3, 4]. Average of middle two: $(2+3)/2 = 2.5$.

Example 3:

- Input: nums = [10, 100, 50]
- Output: 50

Logic :

- Sort the Array: Arrange the elements in ascending order first.
- Check Length: Determine if the number of elements (n) is odd or even.
- Odd Case: If n is odd, the median is the element at the middle index: $\text{nums}[n/2]$.
- Even Case: If n is even, the median is the average of the two middle elements.
- Return: Provide the calculated median value.
-



Q. Rotate the array elements to the left by k positions using the Block Swap Algorithm.

Example 1:

- Input: nums = [1, 2, 3, 4, 5], k = 2
- Output: [3, 4, 5, 1, 2]
- Explanation: The first 2 elements [1, 2] are moved to the end.

Example 2:

- Input: nums = [10, 20, 30], k = 1
- Output: [20, 30, 10]

Example 3:

- Input: nums = [1, 2, 3, 4], k = 4
- Output: [1, 2, 3, 4]

Logic :

- **Divide:** Split the array into two blocks: $A = \text{nums}[0 \dots k-1]$ and $B = \text{nums}[k \dots n-1]$.
- **Compare Sizes:** If the size of block A is equal to B, swap A and B.
- **Recursive Swap:** If A is smaller, divide B into B_l and B_r where B_r size equals A. Swap A and B_r and repeat.
- **Handle Remainders:** If B is smaller, divide A into A_l and A_r where A_l size equals B. Swap A_l and B and repeat.
- **Stop:** Continue until the sizes match and all swaps are complete.



C++

```
void rearrange(vector<int>& nums) {  
    int n = nums.size();  
    sort(nums.begin(), nums.end());  
    reverse(nums.begin() + n / 2, nums.end());  
}
```

Python

```
def rearrange(nums):  
    nums.sort()  
    n = len(nums)  
    mid = n // 2  
    return nums[:mid] + nums[mid:][::-1]
```

Java

```
public void rearrange(int[] nums) {  
    Arrays.sort(nums);  
    int n = nums.length;  
    int left = n / 2, right = n - 1;  
    while (left < right) {  
        int temp = nums[left];  
        nums[left++] = nums[right];  
        nums[right--] = temp;  
    }  
}
```

Q. Rearrange the array such that the first half is in increasing order and the second half is in decreasing order.

Example 1:

- Input: nums = [8, 7, 1, 6, 5, 9]
- Output: [1, 5, 6, 9, 8, 7]
- Explanation: Sorted array is [1,5,6,7,8,9]. First half [1,5,6] is ascending, second half [7,8,9] is reversed.

Example 2:

- Input: nums = [4, 2, 7]
- Output: [2, 7, 4]

Example 3:

- Input: nums = [1, 2, 3, 4]
- Output: [1, 2, 4, 3]

Logic :

- Sort: First, sort the entire array in ascending order.
- Identify Midpoint: Find the middle index of the sorted array.
- Keep First Half: The elements from index 0 to the middle remain in ascending order.
- Reverse Second Half: Flip the order of elements from the middle point to the end of the array.
- Combine: Join both halves to get the final result.



C++

```
void countFreq(vector<int>& nums) {  
    unordered_map<int, int> freq;  
    for (int x : nums) freq[x]++;  
    for (auto it : freq) cout << it.first << ": " << it.second << endl;  
}
```

Python

```
def countFreq(nums):  
    freq = {}  
    for x in nums:  
        freq[x] = freq.get(x, 0) + 1  
    return freq
```

Java

```
public void countFreq(int[] nums) {  
    HashMap<Integer, Integer> map = new HashMap<>();  
    for (int x : nums) {  
        map.put(x, map.getOrDefault(x, 0) + 1);  
    }  
    System.out.println(map);  
}
```

Q. Given an array, count how many times each distinct element appears in the array.

Example 1:

- Input: nums = [10, 5, 10, 15, 10, 5]
- Output: 10: 3, 5: 2, 15: 1
- Explanation: Element 10 appears 3 times, 5 appears 2 times, and 15 appears once.

Example 2:

- Input: nums = [2, 2, 3, 4, 4, 2]
- Output: 2: 3, 3: 1, 4: 2

Example 3:

- Input: nums = [1, 1, 1]
- Output: 1: 3

Logic :

- **Initialize a Map:** Create an empty hash map (dictionary) to store {element: count} pairs.
- **Iterate:** Loop through the array one element at a time.
- **Check Map:** If the element is already in your map, increment its count by 1.
- **Add to Map:** If it's not in the map, save the element as a key with a value (count) of 1.
- **Repeat:** Move to the next number until all elements are processed.



C++

```
void reverseArray(vector<int>& nums) {  
    int left = 0, right = nums.size() - 1;  
    while (left < right) {  
        swap(nums[left++], nums[right--]);  
    }  
}
```

Python

```
def reverseArray(nums):  
    left, right = 0, len(nums) - 1  
    while left < right:  
        nums[left], nums[right] = nums[right], nums[left]  
        left += 1  
        right -= 1
```

Java

```
public void reverseArray(int[] nums) {  
    int left = 0, right = nums.length - 1;  
    while (left < right) {  
        int temp = nums[left];  
        nums[left++] = nums[right];  
        nums[right--] = temp;  
    }  
}
```

Q. Given an array, reverse the order of elements so that the last becomes first and the first becomes last.

Example 1:

- Input: `nums = [5, 4, 3, 2, 1]`
- Output: `[1, 2, 3, 4, 5]`
- Explanation: All elements are flipped to their opposite positions.

Example 2:

- Input: `nums = [10, 20]`
- Output: `[20, 10]`

Example 3:

- Input: `nums = [1, 1, 2]`
- Output: `[2, 1, 1]`

Logic :

- **Two Pointers:** Initialize two variables: `left = 0` and `right = n - 1`.
- **Iterate:** Loop as long as `left < right`.
- **Swap:** Swap the values stored at `nums[left]` and `nums[right]`.
- **Move Pointers:** Increment `left` by 1 and decrement `right` by 1.
- **Repeat:** Continue until the pointers meet or cross in the middle.



Java

```
public class Main {  
    public static void getElements(int[] arr) {  
        int small = Integer.MAX_VALUE;  
        int second_small = Integer.MAX_VALUE;  
        int large = Integer.MIN_VALUE;  
        int second_large = Integer.MIN_VALUE;  
  
        for (int x : arr) {  
            // Second Smallest Logic  
            if (x < small) {  
                second_small = small;  
                small = x;  
            } else if (x < second_small && x != small) {  
                second_small = x;  
            }  
  
            // Second Largest Logic  
            if (x > large) {  
                second_large = large;  
                large = x;  
            } else if (x > second_large && x != large) {  
                second_large = x;  
            }  
        }  
  
        System.out.println("Second Smallest: " + second_small);  
        System.out.println("Second Largest: " + second_large);  
    }  
}
```

Python

```
def getElements(arr):
    small = second_small = float('inf')
    large = second_large = float('-inf')

    for x in arr:
        # Second Smallest Logic
        if x < small:
            second_small = small
            small = x
        elif x < second_small and x != small:
            second_small = x

        # Second Largest Logic
        if x > large:
            second_large = large
            large = x
        elif x > second_large and x != large:
            second_large = x

    return second_small, second_large
```

C++

```
#include <iostream>
#include <vector>
#include <limits>

using namespace std;

void getElements(vector<int>& arr) {
    if (arr.size() < 2) return;

    int small = INT_MAX, second_small = INT_MAX;
    int large = INT_MIN, second_large = INT_MIN;

    for (int x : arr) {
        // Second Smallest Logic
        if (x < small) {
            second_small = small;
            small = x;
        } else if (x < second_small && x != small) {
            second_small = x;
        }

        // Second Largest Logic
        if (x > large) {
            second_large = large;
            large = x;
        } else if (x > second_large && x != large) {
            second_large = x;
        }
    }

    cout << "Second Smallest: " << second_small << endl;
    cout << "Second Largest: " << second_large << endl;
}
```

CODERMIND

Q. Find the second smallest and second largest elements in an array without sorting it.

Example 1 :

Input: [1, 7, 4, 7, 1, 5]

Output: Second Smallest: 2, Second Largest: 5

Example 2 :

Input: [1, 2, 4, 7, 7, 6]

Output: Second Smallest: 2, Second Largest: 6

Example 3 :

Input: [1, 8, 4, 7]

Output: Second Smallest: 4, Second Largest: 7

Logic :

- For Second Largest: Keep two variables, large and second_large. Initialize both to a very small value.
- Compare: If the current number is bigger than large, update second_large = large and large = current.
- Else If: If current is smaller than large but bigger than second_large, update second_large.
- Repeat: Use similar logic for the second smallest.



C++

```
int findMax(vector<int>& nums) {  
    int maxV = nums[0];  
    for(int x : nums) {  
        if(x > maxV) maxV = x;  
    }  
    return maxV;  
}
```

Python

```
# Python  
def findMax(nums):  
    return max(nums)  
  
# Java  
public int findMax(int[] nums) {  
    int max = nums[0];  
    for(int n : nums) if(n > max) max = n;  
    return max;  
}
```

Q. Given an array of integers, find and return the largest element.

Example 1 :

Input: nums = [10, 20, 4, 45, 99]

Output: 99

Example 2 :

Input: nums = [5, 14, 7, 66, 9]

Output: 66

Example 3 :

Input: nums = [1, 7, 88, 19]

Output: 88

Logic :

- Initialize: Set max_val to the first element: nums[0].
- Iterate: Traverse the array.
- Update: If the current element is greater than max_val, update max_val.



C++

```
int findMin(vector<int>& nums) {  
    int minVal = nums[0];  
    for (int num : nums) {  
        if (num < minVal) {  
            minVal = num;  
        }  
    }  
    return minVal;  
}
```

Python

```
def find_min(nums):  
    # Uses the built-in C-optimized min function  
    return min(nums)
```

Java

```
public int findMin(int[] nums) {  
    int minVal = nums[0];  
    for (int num : nums) {  
        if (num < minVal) {  
            minVal = num;  
        }  
    }  
    return minVal;  
}
```

Q. Given an array of integers, find and return the smallest element present in the array.

Example 1:

Input: nums = [5, 8, 1, 4, 7]

Output: 1

Explanation: Among all the elements, 1 is the minimum value.

Example 2:

Input: nums = [3,2,4]

Output: 2

Example 3:

Input: nums = [3,3]

Output: 3

Logic :

- Initialize: Assume the first element of the array is the smallest. Create a variable min_val = nums[0].
- Iterate: Loop through the rest of the array starting from the second element.
- Compare: For each element, if it is smaller than our current min_val, update min_val to this new number.
- Return: Once the loop finishes, min_val will hold the smallest number.



Python

```
def twoSum(nums, target):
    # Dictionary to store: {value: index}
    prev_map = {}

    for i, n in enumerate(nums):
        diff = target - n

        # If the complement exists in our map, we found the pair
        if diff in prev_map:
            return [prev_map[diff], i]

        # Otherwise, store the current number and its index
        prev_map[n] = i
```

Java

```
import java.util.HashMap;
import java.util.Map;

class Solution {
    public int[] twoSum(int[] nums, int target) {
        // Map to store: <Value, Index>
        Map<Integer, Integer> prevMap = new HashMap<>();

        for (int i = 0; i < nums.length; i++) {
            int num = nums[i];
            int diff = target - num;

            // If the complement exists in our map, return the indices
            if (prevMap.containsKey(diff)) {
                return new int[] { prevMap.get(diff), i };
            }

            // Otherwise, add current number and index to the map
            prevMap.put(num, i);
        }

        // Return empty array if no solution is found (though the problem guarantees a solution)
        return new int[] {};
    }
}
```

Q. Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`.

You may assume that each input would have exactly one solution, and you may not use the same element twice.

You can return the answer in any order.

Example 1:

Input: `nums = [2,7,11,15]`, `target = 9`

Output: `[0,1]`

Explanation: Because `nums[0] + nums[1] == 9`, we return `[0, 1]`.

Example 2:

Input: `nums = [3,2,4]`, `target = 6`

Output: `[1,2]`

Example 3:

Input: `nums = [3,3]`, `target = 6`

Output: `[0,1]`

Logic :

- Initialize a Map: Create an empty hash map (dictionary) to store {value: index} pairs.
- Iterate: Loop through the array one element at a time.
- Calculate Complement: For the current number, calculate `complement = target - current_number`.
- Check Map: * If the complement is already in your map, you've found the pair! Return the index of the complement and the current index.
- If it's not in the map, save the current number and its index into the map.
- Repeat: Move to the next number until the pair is found.



Index

IV. String Manipulation

- Write a program to find a word in a given string which has the highest number of repeated letters
- Change case of each character in a string
- Write a program to find a substring within a string (Display starting position)

V. Story Based

21 Story Based Questions

Use This Coding Sheet Smartly

Focus on logic over syntax. Follow this simple system:

Think Before You Code

- Dry Run with sample input on paper.
- Write pseudocode in simple English.
- If it works on paper, it will work in code.

Solve in your main language.

Translate into another language to build versatility.

Start with Brute Force (basic working logic).

Improve to Optimized Solution (better time complexity).

💡 *2 deep problems daily > 20 copied ones.
Master logic. The rest follows.*

Index

III. Number System Conversions

- Convert Binary to Decimal
- Convert Binary to Octal
- Decimal to Binary conversion
- Convert decimal to octal
- Convert octal to binary
- Convert octal to decimal
- Convert digits/numbers to words

IV. String Manipulation

- Count number of vowels, consonants, spaces in String
- Remove all vowels from the string
- Remove spaces from a string
- Remove characters from a string except alphabets
- Remove brackets from an algebraic expression
- Sum of the numbers in a String
- Capitalize first and last character of each word
- Calculate frequency of characters in a string
- Find Non-repeating characters of a String
- Check if two strings are anagram of each other
- Count common sub-sequence in two strings
- Remove characters from first string present in the second string
- Change every letter with the next lexicographic alphabet in the given string
- Write a program to find the largest word in a given string

Index

I. Array & Matrix Operations

- Pascal's Triangle
- Employee Bonus (SQL/Logic)
- Same Tree (Binary Tree)
- Largest Rectangle in Histogram
- Reverse Linked List
- Set Matrix Zeroes
- Spiral Matrix
- Trapping Rain Water
- Palindrome Partitioning IV

II. Mathematical & Number Theory

- Prime numbers in a given range
- Maximum and Minimum digit in a number
- Print all prime factors of the given number
- Check if a number is a strong number or not
- Check if a Number is Automorphic
- GCD of two numbers
- LCM of two numbers
- Check if a number is Harshad number
- Check if the number is abundant number or not
- Permutations in which N people can occupy R seats in a classroom
- Replace all 0s with 1s in a given integer
- Program to find roots of a Quadratic Equation

Index

I. Array & Matrix Operations

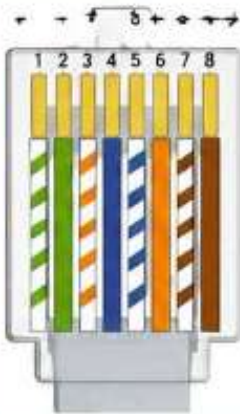
- Two Sum
- Find the smallest number in an array
- Find the largest number in an array
- Second Smallest and Second Largest element in an array
- Reverse a given array
- Count frequency of each element in an array
- Rearrange array in increasing-decreasing order
- Calculate sum of the elements of the array
- Rotate array by K elements - Block Swap Algorithm
- Average of all elements in an array
- Find the median of the given array
- Remove duplicates from a sorted array
- Remove duplicates from unsorted array
- Find all repeating elements in an array
- Find all non-repeating elements in an array
- Find all symmetric pairs in array
- Maximum product subarray in an array
- Replace each element of the array by its rank in the array
- Sorting elements of an array by frequency
- Rotation of elements of array - left and right
- Finding equilibrium index of an array
- Check if Array is a subset of another array or not
- Binary Search
- Search a 2D Matrix
- Path with Maximum Probability
- Students and Examinations (SQL/Logic)
- Next Permutation

RJ45 COLOR CODING

Ethernet Cable Wiring Standards

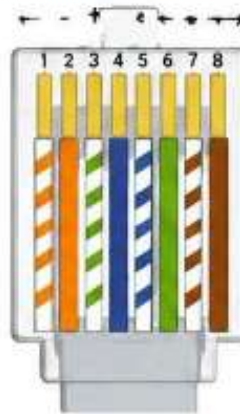


T568A (WIRING STANDARD)



PIN	COLOR	DESCRIPTION
1	White/Green	White/Green
2	Green	Green
3	White/Orange	White/Orange
4	Blue	Blue
5	White/Blue	White/Blue
6	Orange	Orange
7	White/Brown	White/Brown
8	Brown	Brown

T568B (WIRING STANDARD) (MOST COMMONLY USED)



PIN	COLOR	DESCRIPTION
1	White/Orange	White/Orange
2	Orange	Orange
3	White/Green	White/Green
4	Blue	Blue
5	White/Blue	White/Blue
6	Green	Green
7	White/Brown	White/Brown
8	Brown	Brown

TYPES OF ETHERNET CABLES

STRAIGHT-THROUGH CABLE

T568B



Both ends
same standard
(T568B - T568B)
or (T568A - T568A)

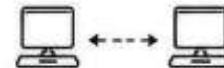
T568B



USED FOR:
PC ↔ Switch, Router ↔ Switch,
Hub ↔ Switch

CROSSOVER CABLE

T568A



Different standard
on each end
(T568A - T568B)

T568B



USED FOR:
PC ↔ PC, Switch ↔ Switch,
Router ↔ Router
(Older devices - Auto MDI/MDIX modern devices)



**EASY WAY TO
REMEMBER T568B**



Orange - Orange - Green - Blue - Blue - Green - Brown - Brown

RJ45 PINOUT (FRONT VIEW)



Always hold the connector
with the clip at the back
and the pins facing up.



NOTE:

- T568B is the most widely used standard.
- Straight-through cable is used in most modern networks.
- Crossover cable is rarely required today due to Auto MDI/MDIX support in devices.



CAT5e
CAT6
CAT6A





MULTICAST VS ANYCAST



DELIVERING DATA TO MANY: GROUPS vs NEAREST

MULTICAST – One to Many (Group Delivery)

Data is sent from one source to a group of receivers that have joined a multicast group.



Example Use Cases

- IP TV, Live Streaming
- Online Gaming
- Financial Data Feeds

ANYCAST – One to Nearest (Location-Based Delivery)

Data is sent from one source to the nearest available node (closest in terms of routing).



Example Use Cases

- DNS Root Servers
- CDN (Content Delivery)
- Load Balancing Services

FEATURE	MULTICAST	VS	ANYCAST
COMMUNICATION TYPE	One-to-Many (Group)	VS	One-to-Nearest (Node)
ADDRESS RANGE	224.0.0.0 to 239.255.255.255 (IPv4) FF00::/8 (IPv6)	VS	Unicast IP address from normal range
DELIVERY BASIS	To all group members that joined	VS	To the nearest available node
ROUTING	Uses Multicast Routing Protocols (IGMP, PIM, etc.)	VS	Uses Unicast Routing Protocols (BGP, OSPF, etc.)
RECEIVERS	Multiple receivers can get the data simultaneously	VS	Only one (nearest) node receives the data
BANDWIDTH USAGE	Efficient – one stream for all receivers	VS	Efficient – routed to nearest node only
SCALABILITY	Scales well for large number of receivers in same group	VS	Scales based on number of anycast nodes geographically
FAULT TOLERANCE	If source fails, all receivers are affected	VS	If one node fails, traffic goes to next nearest
TYPICAL USE CASES	IPTV, Live streaming, Stock tickers, Online gaming, Video conferencing	VS	DNS, CDN, Global load balancing, Low-latency services
ADDRESS SEEN BY CLIENT	Multicast group address	VS	Same anycast IP, but different nearest node
EXAMPLE ADDRESS	239.1.1.1 (IPv4)	VS	8.8.8.8 (Google DNS Anycast)

KEY TAKEAWAY



Use **MULTICAST** when you want to send data to multiple receivers that are part of the same group.



Use **ANYCAST** when you want to send data to the nearest available node for better performance and reliability.



Chuyển Mạch Ethernet: Cách Switch "Suy Nghĩ"

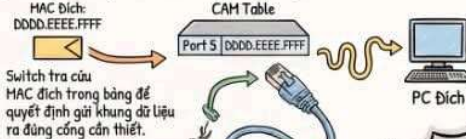
Giải thích cơ chế học hỏi, chuyển tiếp khung dữ liệu và các phương thức hoạt động cốt lõi của Switch Ethernet trong mạng LAN.

CƠ CHẾ BẢNG ĐỊA CHỈ MAC (BẢNG CAM)

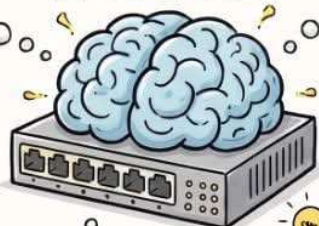
1 Học (Learning) từ địa chỉ Nguồn



2 Chuyển tiếp (Forwarding) theo địa chỉ Đích



ETHERNET SWITCH (BỘ CHUYỂN MẠCH)



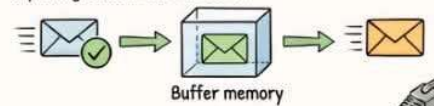
Chuyển tiếp tràn ngập (Flooding)



CÁC PHƯƠNG THỨC CHUYỂN TIẾP KHUNG

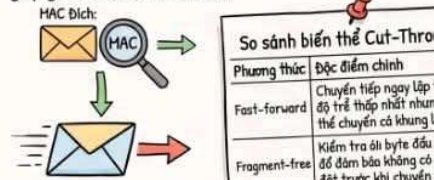
Lưu và Chuyển tiếp (Store-and-Forward)

Nhận toàn bộ khung, kiểm tra lỗi (FCS) trước khi gửi đi; là phương thức ưu tiên của Cisco.



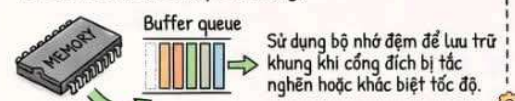
Chuyển tiếp ngay (Cut-Through)

Gửi khung ngay sau khi đọc xong địa chỉ MAC đích, giúp giảm thiểu độ trễ tối đa.



So sánh biến thể Cut-Through	
Phương thức	Đặc điểm chính
Fast-forward	Chuyển tiếp ngay lập tức, độ trễ thấp nhất nhưng có thể chuyển cả khung lỗi.
Fragment-free	Kiểm tra 64 byte đầu tiên để đảm bảo không có xung đột trước khi chuyển tiếp.

Đệm bộ nhớ (Memory Buffering)

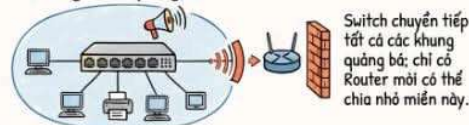


QUẢN LÝ MIỀN XUNG ĐỘT & QUẢNG BÁ

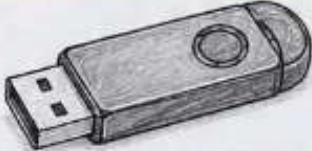
Loại bỏ Miền xung đột (Collision Domains)



Mở rộng Miền quảng bá (Broadcast Domains)



DIFFERENT STORAGE DEVICES

S.No.	STORAGE DEVICE	DRAWING	USE
1.	<u>HDD</u>		Data ko permanently store karne ke liye use hota hai.
2.	<u>SSD</u>		Fast storage hai, computer ko jaldi boot aur files jaldi open karta hai.
3.	<u>Pen Drive</u>		Files ko store aur ek device se dusre device me transfer karne ke liye use hota hai.
4.	<u>Memory Card</u>		Mobile, camera aur other devices me data store karne ke liye use hota hai.
5.	<u>CD</u>		Data, music ya software store karne ke liye optical disc use hoti hai.
6.	<u>DVD</u>		CD se zyada data store karne ke liye use hoti hai.
7.	<u>Blu-ray Disc</u>		High-quality video aur large data store karne ke liye use hoti hai.
8.	<u>External HDD</u>		Extra storage aur backup ke liye computer ke bahar connect karke use hota hai.

Linux Administration

Sun Networking Academy

1. System Information

- `uname -a` → Kernel and system info
- `hostnamectl` → Hostname and OS details
- `uptime` → System uptime and load
- `df -h` → Disk space usage
- `free -m` → Memory usage
- `lscpu` → CPU information

2. User Management

- `useradd <user>` → Add a new user
- `passwd <user>` → Set password
- `usermod -aG <group> <user>` → Add user to group
- `userdel -r <user>` → Delete user and home
- `id <user>` → User and group info
- `whoami` → Current logged in user
- `who` → Logged in users

Sun Networking Academy

3. File and Directory Management

- `ls -l` → List files (long format)
- `cd <dir>` → Change directory
- `pwd` → Print working directory
- `mkdir <dir>` → Create directory
- `rm <file>` → Remove file
- `rm -r <dir>` → Remove directory recursively
- `cp <src> <dst>` → Copy file or directory
- `mv <src> <dst>` → Move or rename
- `chmod <mode> <file>` → Change permissions
- `chown <user>:<group> <file>` → Change owner

4. Package Management (Debian/Ubuntu)

- `apt update` → Update package list
- `apt upgrade` → Upgrade packages
- `apt install <pkg>` → Install package
- `apt remove <pkg>` → Remove package
- `apt search <pkg>` → Search package
- `dpkg -l` → List installed packages

5. Service Management (systemd)

- `systemctl status <service>` → Check status
- `systemctl start <service>` → Start service
- `systemctl stop <service>` → Stop service
- `systemctl restart <service>` → Restart service
- `systemctl enable <service>` → Enable on boot
- `systemctl disable <service>` → Disable on boot

Sun Networking Academy

6. Network Management

- `ip a` → Show IP addresses
- `ip r` → Show routing table
- `ping <host>` → Test connectivity
- `ss -tuln` → Show listening ports
- `netstat -tuln` → Network connections
- `nmcli device status` → Network device status

7. Log Management

- `journalctl -xe` → View system logs
- `tail -f /var/log/syslog` → Monitor syslog
- `dmesg | less` → Kernel messages

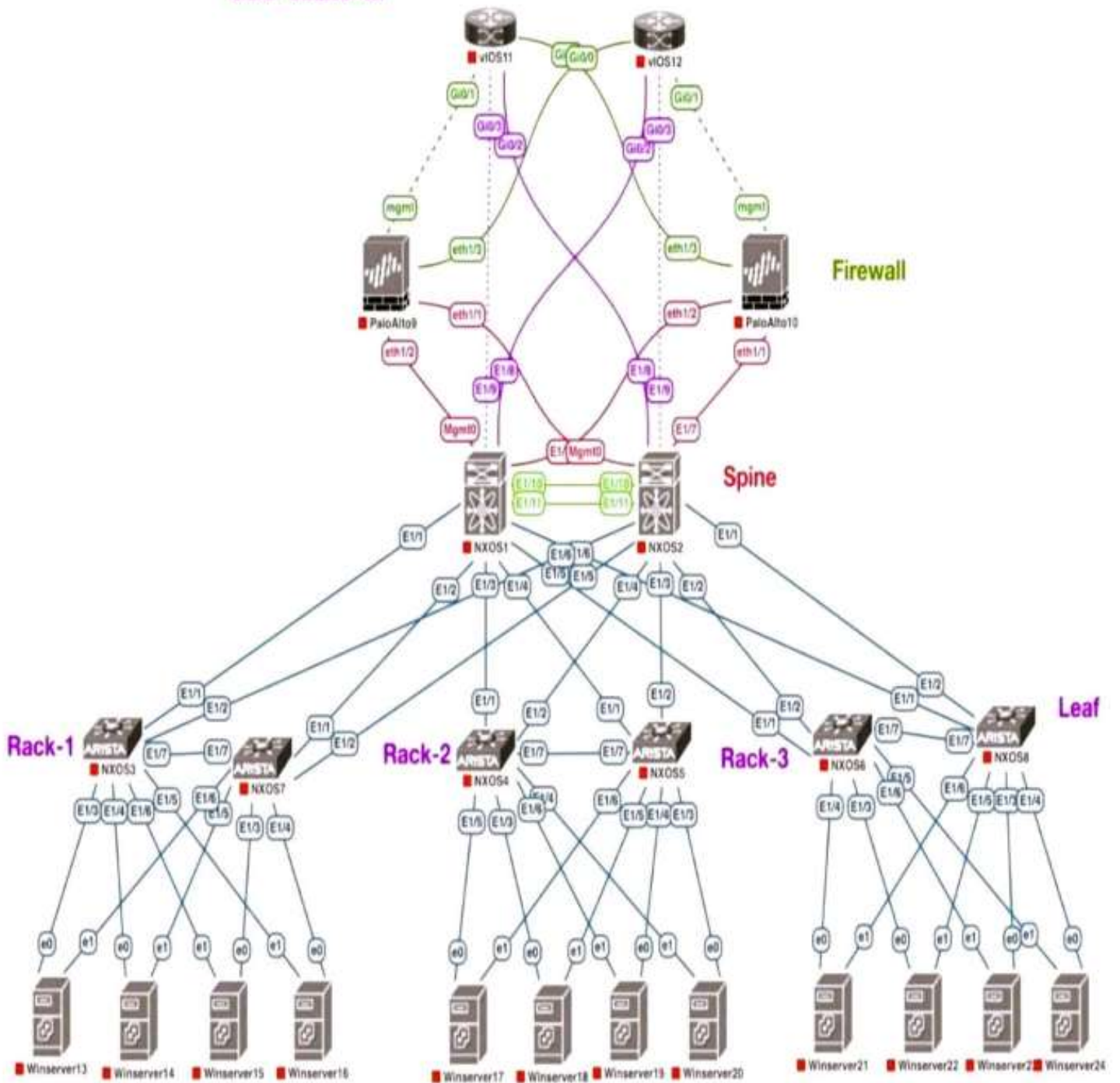
8. Useful Commands

Sun Networking Academy

- | | | |
|--|---|---|
| • <code>clear</code> → Clear terminal | • <code>cat <file></code> → View file content | • <code>find / -name <file></code> → Find file |
| • <code>history</code> → Command history | • <code>less <file></code> → View file page by page | • <code>man <command></code> → Manual page |
| • <code>sudo !!</code> → Repeat last command with sudo | • <code>grep <text> <file></code> → Search text in file | • <code>which <command></code> → Command location |

CCNP DataCenter

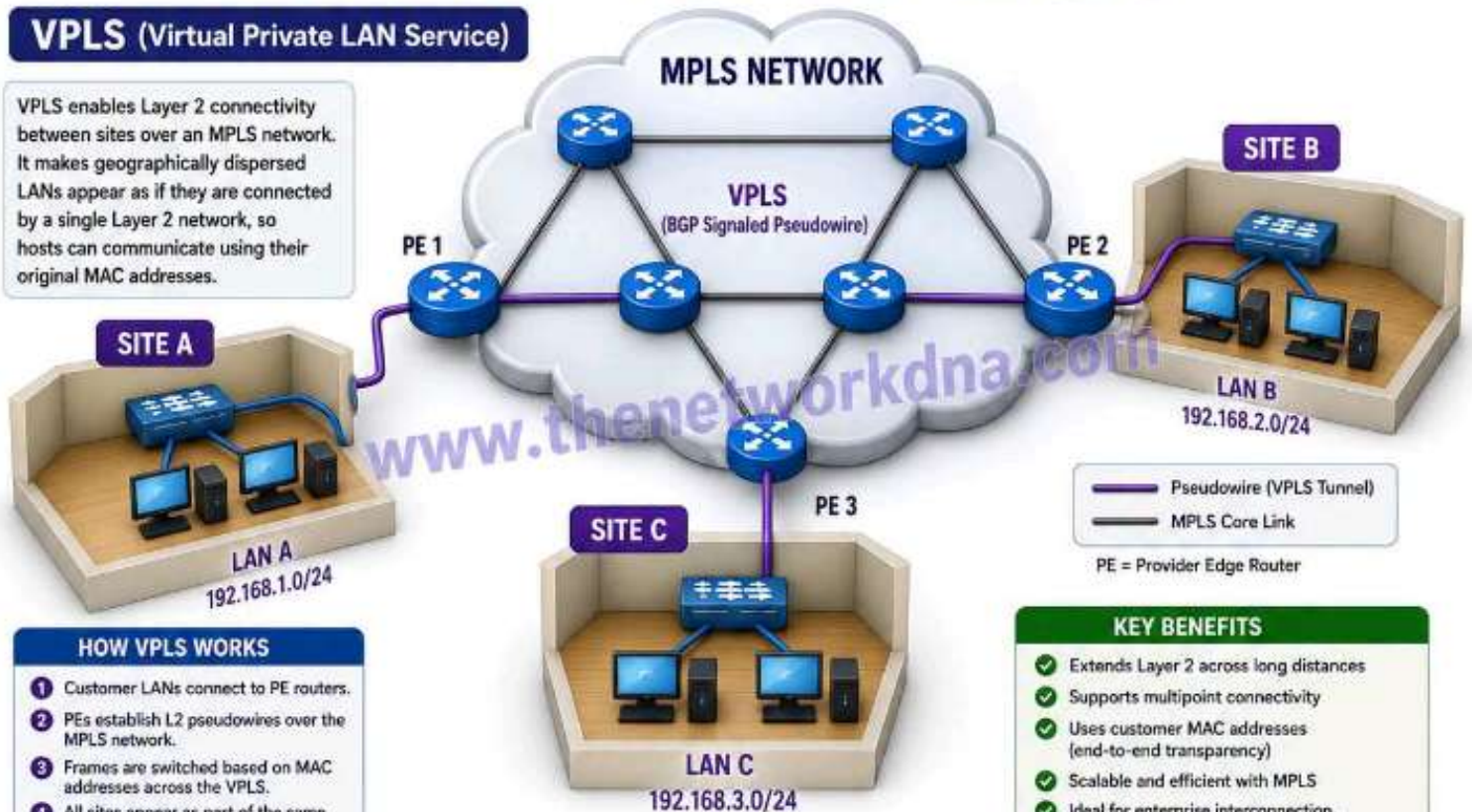
WAN/Internet



WHAT IS VPLS?

VPLS (Virtual Private LAN Service)

VPLS enables Layer 2 connectivity between sites over an MPLS network. It makes geographically dispersed LANs appear as if they are connected by a single Layer 2 network, so hosts can communicate using their original MAC addresses.



HOW VPLS WORKS

- 1 Customer LANs connect to PE routers.
- 2 PEs establish L2 pseudowires over the MPLS network.
- 3 Frames are switched based on MAC addresses across the VPLS.
- 4 All sites appear as part of the same Layer 2 domain.

KEY BENEFITS

- ✓ Extends Layer 2 across long distances
- ✓ Supports multipoint connectivity
- ✓ Uses customer MAC addresses (end-to-end transparency)
- ✓ Scalable and efficient with MPLS
- ✓ Ideal for enterprise interconnection, data center and cloud services

EXAMPLE: L2 CONNECTIVITY BETWEEN SITES



VPLS USE CASES

- Enterprise WAN**
Connect branch offices as if on the same LAN
- Data Center Interconnect**
Extend Layer 2 between data centers
- Cloud Connectivity**
Provide seamless L2 access to cloud services
- Service Provider Services**
Deliver scalable Layer 2 services to customers



What is VPLS?

A Layer 2 VPN service over MPLS that connects multiple sites into one virtual LAN.



How it Works

PE routers use pseudowires over MPLS to bridge Layer 2 frames between sites.



Key Technology

BGP Signaled Pseudowire (Martini) for control plane signaling.



Result

Seamless Layer 2 connectivity with MAC address transparency across the network.

BRANCH OFFICE ARCHITECTURE

www.thenetworkdna.com

POWER DISTRIBUTION



EDGE FIREWALL (FORTINET)



CORE / DISTRIBUTION SWITCH (ARUBA)



www.thenetworkdna.com

ACCESS SWITCHES (ARUBA)



CABLING & PATCH PANELS



SECURITY
Built-in protection
against threats



RELIABILITY
High availability
and redundancy



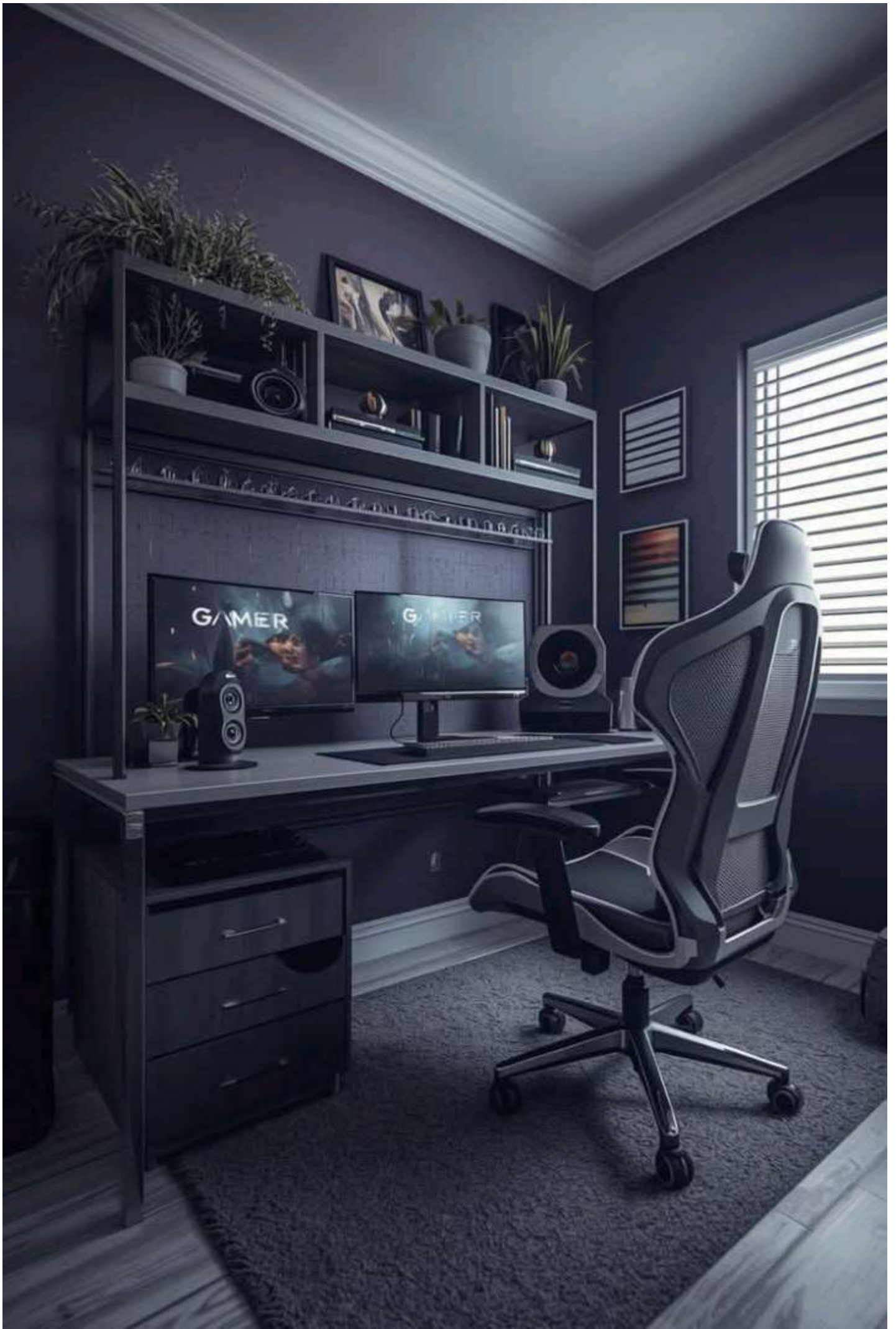
PERFORMANCE
Optimized for speed
and efficiency



MANAGEABILITY
Easy monitoring
and control



VALUE
Cost-effective
solutions



What is a Formatter Board?

A formatter board (main board) is the main electronic circuit board in a printer that controls all printer functions and manages communication between parts.

FORMATTER BOARD (MAIN BOARD)

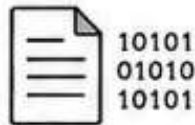


CONTROLS PRINTER OPERATIONS



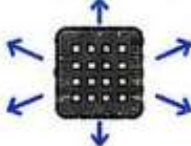
Manages and coordinates all functions of the printer.

PROCESSES PRINT DATA



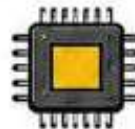
Receives data from the computer and processes it before printing.

COMMUNICATES WITH PARTS



Sends commands to parts like the motor, sensor, heater, and more.

STORES SETTINGS & FIRMWARE



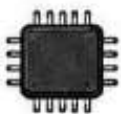
Stores firmware and settings in memory (ROM/Flash).

CONNECTS INTERFACES



Provides connections such as USB, Network, or Parallel port.

Common Components on a Formatter Board



Processor (IC)



Memory (RAM/ROM)



Capacitors



Crystal (Clock)



Connectors



Flash Memory



Battery (CMOS)



Voltage Regulator

Explanation

- Controls Printer Operations - Manages and runs all activities of the printer.
- Processes Print Data - Takes data from the computer and prepares it for printing.
- Communicates with Parts - Sends and receives signals between the board and other parts.
- Stores Settings & Firmware - Keeps the printer's program and settings safely.
- Connects Interfaces - Allows the printer to connect with computers or networks.



The formatter board is like the brain of the printer. If it fails, the printer may not work at all.

COMPLETE NVR SETUP DIAGRAM

STEP BY STEP CONNECTION GUIDE



RJ45 T568A vs T568B

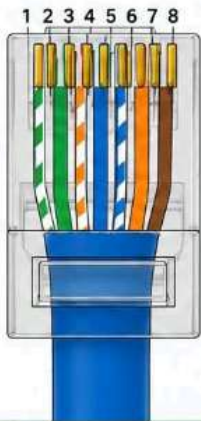
WIRING DIAGRAM

Both T568A and T568B wiring standards work perfectly for Ethernet networks.
Use the same standard on both ends of the cable for straight-through connections.

T568A WIRING STANDARD

PIN	WIRE COLOR	DESCRIPTION
1	White / Green	Transmit +
2	Green	Transmit -
3	White / Orange	Receive +
4	Blue	Unused
5	White / Blue	Unused
6	Orange	Receive -
7	White / Brown	Unused
8	Brown	Unused

RJ45 PINOUT (FRONT VIEW)



KEY DIFFERENCE

The only difference between T568A and T568B is the position of the **Green** and **Orange** pairs.



Pins 1 & 2 are swapped

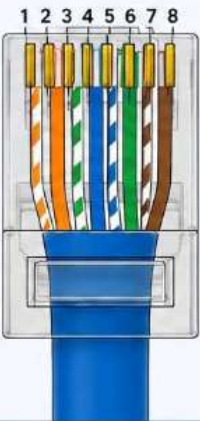


Pins 3 & 6 are swapped

T568B WIRING STANDARD

PIN	WIRE COLOR	DESCRIPTION
1	White / Orange	Transmit +
2	Orange	Transmit -
3	White / Green	Receive +
4	Blue	Unused
5	White / Blue	Unused
6	Green	Receive -
7	White / Brown	Unused
8	Brown	Unused

RJ45 PINOUT (FRONT VIEW)



WHEN TO USE?



STRAIGHT-THROUGH CABLE

Use the same standard on both ends (T568A - T568A or T568B - T568B)



CROSSOVER CABLE

Use T568A on one end and T568B on the other end

TIPS

- ✓ Always use the same standard on both ends for straight-through cables.
- ✓ Both standards support 10/100/1000 Mbps Ethernet speeds.
- ✓ Choose one standard and be consistent.



Reliable Connection



High Performance



Universal Standard

IT SUPPORT TROUBLESHOOTING INTERVIEW

1. Computer Not Turning On

- Check power cable.
- Check power outlet.
- Check power button.
- Check SMPS / charger.
- Remove external devices.
- Restart the PC.
- If still not working, check hardware and escalate.

2. No Internet Connection

- Check network cable / Wi-Fi.
- Run ipconfig.
- Ping gateway.
- Ping Google (8.8.8.8).
- Restart network adapter.
- Restart router.
- Contact network team if needed.

3. Printer Not Printing

- Check power.
- Check USB / Network cable.
- Check paper and toner.
- Clear print queue.
- Restart Print Spooler.
- Reinstall printer driver.

4. Computer Running Slow

- Open Task Manager.
- Check CPU, RAM, Disk usage.
- Close unwanted programs.
- Disable startup apps.
- Scan for viruses.
- Restart computer.
- Upgrade RAM / SSD if needed.

5. Forgot Windows Password

- Verify user identity.
- Reset password in Active Directory.
- Ask user to create a new password.
- Confirm login works.

6. Blue Screen (BSOD)

- Note error code.
- Boot into Safe Mode.
- Update drivers.
- Run sfc /scannow.
- Check RAM and hard disk.
- Restore Windows if needed.

7. Outlook Not Working

- Check internet.
- Restart Outlook.
- Check mailbox size.
- Repair Outlook profile.
- Test webmail.
- Verify mail server.

8. Shared Folder Access Denied

- Check network.
- Verify permissions.
- Check user group.
- Check server availability.
- Test with another account.

9. Wi-Fi Not Connecting

- Turn Wi-Fi ON.
- Restart adapter.
- Forget and reconnect network.
- Update Wi-Fi driver.
- Restart router.
- Test another device.

10. Computer Frozen

- Wait a few seconds.
- Press Ctrl + Shift + Esc.
- End frozen application.
- Restart PC.
- Check Event Viewer.

IMPORTANT COMMANDS

ipconfig	→ Show IP address
ipconfig /all	→ Full network details
ping	→ Check connectivity
tracert	→ Trace network path
nslookup	→ Check DNS
netstat	→ Show network connections
tasklist	→ List running processes
taskkill	→ End process
sfc /scannow	→ Repair system files
DISM	→ Repair Windows image
chkdsk	→ Check disk errors
gpupdate /force	→ Update Group Policy

BASIC TROUBLESHOOTING STEPS

1. Identify the problem.
2. Ask the user questions.
3. Check cables and connections.
4. Test the issue.
5. Apply the solution.
6. Verify the problem is fixed.
7. Document the issue.

INTERVIEW TIP

Remember this line:

"I always start with the basic checks, identify the root cause, test the solution, verify the issue is resolved, and document the resolution."

SANDBOX vs HYPER-V

Which One Should You Use?



WINDOWS SANDBOX



Quick • Safe • Temporary



HYPER-V



Powerful • Flexible • Permanent



Test apps or files safely in an isolated environment.



PURPOSE

Create and run virtual machines for various needs.



Very easy. Enable the feature and run. It's ready to use.



OPERATING SYSTEM

Supports Windows, Linux, and other operating systems.



Runs a clean, temporary Windows environment.



DATA PERSISTENCE

Data and settings are saved until you delete the VM.



Very limited customization.



CUSTOMIZATION

Full control over hardware, settings, and more.



Quick testing, opening unknown files, safe browsing.



BEST FOR

Development, IT labs, server environments, multiple OS testing.



CHOOSE WINDOWS SANDBOX IF:



- You need a quick, disposable Windows environment.
- You want to test unknown apps or files safely.
- You don't need to save your work.

CHOOSE HYPER-V IF:



- You need permanent virtual machines.
- You want to run multiple operating systems.
- You need advanced virtualization features for development, testing, or IT administration.

@Smart AI Tutorial



IN SHORT:

WINDOWS SANDBOX

= Temporary & Secure Testing

HYPER-V

= Full Virtual Machine Platform

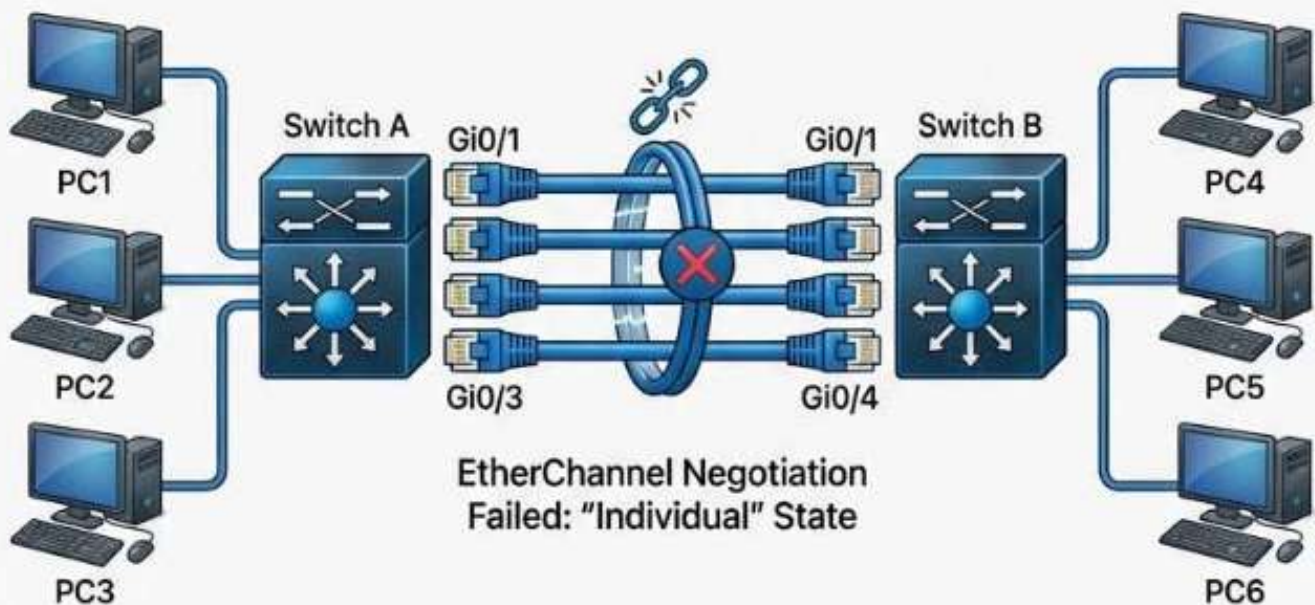
CCNA Quiz

Scenario Challenge

Scenario: Two Cisco switches are connected using four physical links. EtherChannel is configured, but all interfaces remain in the 'Individual' state and traffic is not load balanced.

Question: What should the administrator verify first?

- A. Verify that LACP or PAgP modes match on both switches.
- B. Change the native VLAN on the trunk.
- C. Enable PortFast on all EtherChannel interfaces.
- D. Disable Spanning Tree Protocol.



STP vs RSTP vs MSTP

COMPARISON OF SPANNING TREE PROTOCOLS

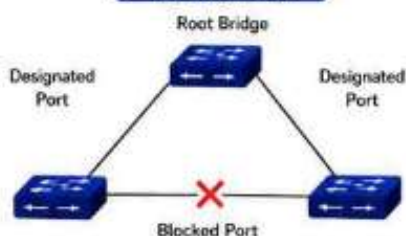


Spanning Tree Protocols prevent Layer 2 loops in networks by blocking redundant paths while keeping a backup path ready.

FEATURE	STP (802.1D)	RSTP (802.1w)	MSTP (802.1s)
STANDARD	IEEE 802.1D	IEEE 802.1w	IEEE 802.1s
CONVERGENCE TIME	30 – 50 seconds	1 – 5 seconds	1 – 5 seconds
TOPOLOGY CHANGE DETECTION	Slow (uses timers)	Fast (uses handshake)	Fast (uses handshake)
LOOP PREVENTION	Yes	Yes	Yes
PORT STATES	5 States (Blocked, Listening, Learning, Forwarding, Disabled)	3 States (Discarding, Learning, Forwarding)	3 States (Discarding, Learning, Forwarding)
VLAN SUPPORT	Per VLAN (PVST)	Per VLAN (PVST+/Rapid-PVST)	Multiple VLANs Mapped to Instances
INSTANCE SUPPORT	One Instance (per VLAN)	One Instance (per VLAN)	Multiple Instances (MST)
BANDWIDTH EFFICIENCY	Lower	Higher	Highest
COMPLEXITY	Simple	Moderate	More Complex
BEST USE CASE	Legacy Networks	Modern Networks (need fast convergence)	Large Networks with many VLANs

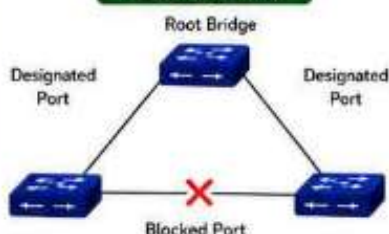
HOW THEY WORK

STP (802.1D)



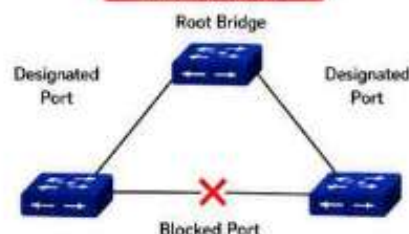
STP blocks redundant links and takes 30–50 seconds to converge.

RSTP (802.1w)



RSTP uses handshake mechanism and converges in 1–5 seconds.

MSTP (802.1s)



MSTP maps multiple VLANs to instances, balancing load and saving resources.